Jasdeep Singh
Jonathan Torres
Saleh Al-Dharhani

PROJECT 3

1.Project Overview

The project that we have decided on is a flight tracker.
It's a progressive web app that targets people who want to check on friends or family's flights departure time, arrival time, ETA, temperature at departure destination and at arrival destination,elevation,where is the plane currently on the globe,etc.

This was our idea that we came up with and decided with that, that this will be our project for project 3. The language that we used reactJS for this app.

Resource	Takeaway	Time log
Chat GPT	API Definitions + Decisions	1:00
rnmapbox	Globe API Reading	0:30
Maplibre	Another Globe API Reading	0:30
Flightradar24	Flight Tracking API	1:00
Weather	Weather API Reading	1:00
Open-meteo	Another Weather API reading	0:30
Claude	Explaining idea and how to implement	2:00

1. PREPARATION PHASE (Complete at least 4 of the following):

- ☑ User Profiles/Stories (target audience, ect…)
- ☑ Features/Requirements Document
- ☐ Relational DB Schema/Diagram
- ☑ App Architecture Document
- ☑ Style Guide / UI Design Document
- ☑ Research log

2. MAIN APP FEATURES (Required)

- Version control to manage code changes
- At least one pure server-side controller
- At least 2 http API calls from the client or a websocket
- Client-Side Data Model Classes
- Well designed Graphical UI/UX

3. ADDITIONAL FEATURES (Demonstrate understanding and use at least 4 of the following):

- ☑ Architecture patterns (e.g., MVC, MVVM, MVP, MVI, etc)
- ☐ Design patterns (e.g., Factory Method, Builder Class, Singleton, Iterator, etc)
- ☑ Persistent Data Storage (either locally or behind an API)
- ☐ Concurrency/multithreading
- ☑ Client/Server data validation/sanitization (verifying email/password & flight num)(Sanitization Denying SQL Inserts or Injections)
- ☐ Server-side rendering
- ☑ 3rd-party APIs/integrations
- ☑ API testing (e.g., curl, insomnia, postman, JaSON)
- ☑ State-handing with asynchronous functions (callbacks vs promises vs async/await)
- ☐ Virtual Machines/Containers

4. Set-up

We used github for pulling and pushing and discord for communication.

We all forked from one main repository (Jon's) and cloned the fork into VScode so we can all work on it locally and individually without it hindering us.

Environment/Development

Environment:

For the environment we decided to use VScode since it has version control which makes it easier to commit, sync, push and pull.

Development:

For our Development, we first started with a simple skeleton, and did not have any log in pages.
We focused on flight API's and Map API's at the start, all UI Colors were black and greys with a simple skeleton mock up.
After coding all of our requests our biggest struggle was verifying which API calls were included in our subscription with flight21, and which calls were not as we were under the impression that we had access to all API Calls.
We used a postman to verify that all the requests are working fully, and we realized there was some function we did not have access to like Airport data in full or flight data in full. On top of that we could not use the sandbox keys that were provided because they provided back mock data which looked like our progress and development was working, but when we used our production key we saw the true flaws and errors.

Apart from this set back from not having access to airport data we ended up hard coding airports with their map positioning so we can have a clean interface showcasing a dotted line of the airplane's current location.

Mid development we switched our weather api to Meteo as it was free and had more features and also realized we did not like Flight21 but had to make use of it since we already paid for it.

Further in development, each change published was discussed and we made sure to separate documents so we would not run into any merge conflicts which turned out flawlessly.

Features:

Progressive web app - You can download this app locally on your device where you can access it without having to use google.

Tracked flights - You can save tracked flights where it will be saved onto an account (stored locally). You can have multiple tracked at once.

Rotating Globe - When tracking flights it projects onto a globe that shows you where that plane coordinates.

Shared Live Link - Give each tracked flight a shareable page or deep link so family/friends can open the same flight quickly.

Issues/Bugs

We encountered a lot of bugs and issues. The front end was fine for the most part. It was mostly with the backend. How much we were calling the apis.

Back-end

We didn't realize that there was this one line of code that rammed out our apis. It wasted our free weather api so we had to use a new one. For our flight tracker it used around two thousand calls. It was calling the apis 10 times a second or something like that
After we changed the weather api it wasnt displaying anything but still using the api tokens.

Front-end

With the design we tried to make it as simplistic as possible. So we kept changing the design until we stuck with one design. As mentioned before we started with a skeleton of a black and grey themes. Front End had minimal problems. Here are the problems we encountered. The plane was always facing east, so we fixed it so it faces the arrival destination. The departure times would be mixed up with planes who already had arrived. Share links did not provide the accurate info at some point in time and had way separate data as if the original API data did not get shared via link. The global map did not display the line across the map correctly, or spun around the earth multiple times before actually getting to its destination due to having the incorrect math calculations of calculating the line between two differences. In some countries the plane would act as if it is barely arriving to the departure airport instead of the arrival airport. There was lots of bugs dealing with two API’s and one that we were forced to use because we had already forked over the money for it. Flightradar24 offers the least amount of full service api details

extrusions ive seen yet. Through it all we hard coded the airports and we made use to a functional production ready build!

Design

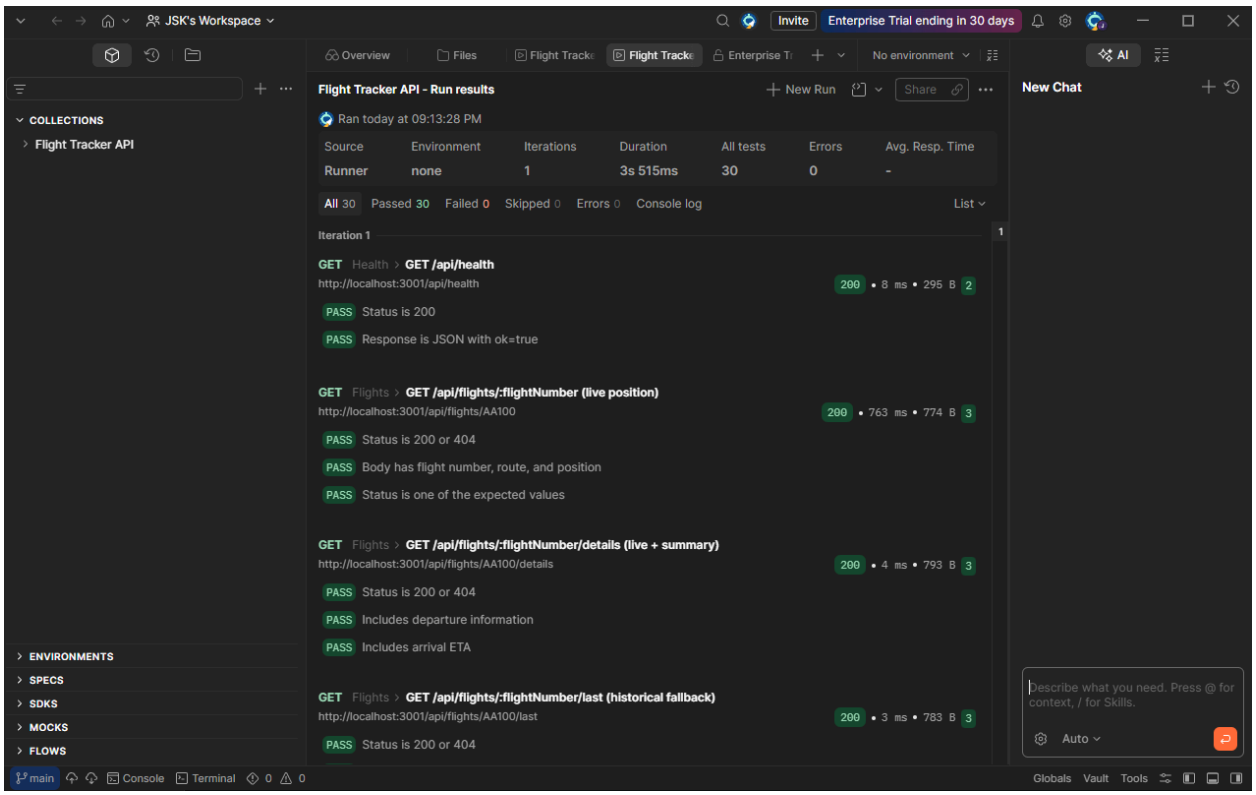
We started with a basic skeleton layout and a simple globe map. Early on we added the login and sign up pages, then went through a bit of tweaking text colors and fixing alignment issues. The plane icon on the map was pointing the wrong direction so we fixed that, then improved how it looks overall. We changed the globe's background to a dark space and stars look to make it feel more like a flight tracker. The weather cards and flight detail info cards got restyled to look cleaner and more organized. We added a runway photo as the background on the main search page to give it a stronger first impression. Then we added an airplane photo as the background behind the sign in and sign up boxes. Most recently we cleaned up the CSS file by removing duplicate and redundant styles to make it shorter without changing how anything looks.

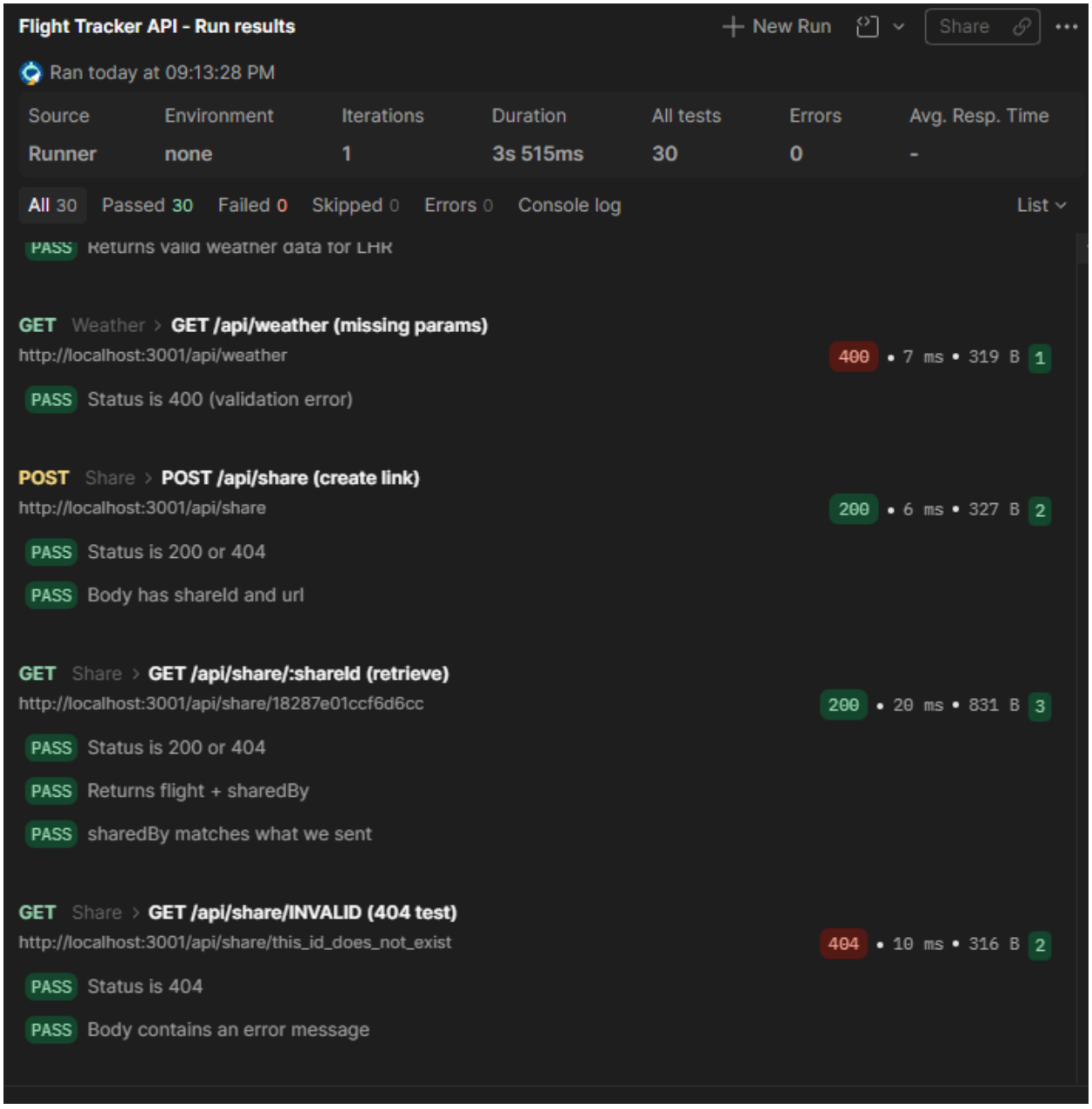
Deployment

We deployed the Flight Tracker app to the internet using a free platform called Render, which connects directly to our private GitHub repository. Whenever new code is pushed to the main branch, Render automatically rebuilds and restarts the app with no manual work needed. The app only needed one environment variable to work, which was the FlightRadar24 API key, since the weather API is completely free and requires no key. The Express server handles both the backend API and serves the React frontend in production mode, so everything runs as one single service under one public URL that anyone can visit from any browser.

Post-Man

Below is our post man calls showing that our API tests were successful!




Flight Tracker API - Run results
New Run
Share
Ran today at 09:13:28 PM
Source
Environment
Iterations
Duration
All tests
Errors
Avg. Resp. Time
Runner
none
1
3s 515ms
30
0
-
All 30
Passed 30
Failed 0
Skipped 0
Errors 0
Console log
List
PASS Returns valid weather data for LHR
GET Weather > GET /api/weather (missing params)
http://localhost:3001/api/weather
400 • 7 ms • 319 B 1
PASS Status is 400 (validation error)
POST Share > POST /api/share (create link)
http://localhost:3001/api/share
200 • 6 ms • 327 B 2
PASS Status is 200 or 404
PASS Body has shareId and url
GET Share > GET /api/share/:shareId (retrieve)
http://localhost:3001/api/share/18287e01ccf6d6cc
200 • 20 ms • 831 B 3
PASS Status is 200 or 404
PASS Returns flight + sharedBy
PASS sharedBy matches what we sent
GET Share > GET /api/share/INVALID (404 test)
http://localhost:3001/api/share/this_id_does_not_exist
404 • 10 ms • 316 B 2
PASS Status is 404
PASS Body contains an error message

https://cmps-3390-flight-tracker.onrender.com/